WEEK 4 · LESSON 24 OF 24 · HANDOUT

Final Project: An End-to-End Model and Certificate

Study time: **about 120 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=nKW8Ndu7Mjw — open the lesson online to watch inside the course.

🎓 **Go deeper:** freeCodeCamp: scikit-learn — full course — https://www.youtube.com/watch?v=pqNCD_5r0IU

🎓 **Go deeper:** SciPy 2018 tutorial: Machine Learning with scikit-learn, Part 1 (scikit-learn core developers) — https://www.youtube.com/watch?v=4PXAztQtoTg

Your goal in this lesson: Combine everything in the course — preparation, several algorithms, cross-validation, tuning, honest testing, fairness, saving and documentation — in one professional project, and earn your certificate.

Choose your track

- **Track A — Kigali rents (regression).** The full starter code is below.
- **Track B — Micro-loans (classification).** Use the loans data from Week 3, with the region audit from Lesson 22 and a cost-based threshold.
- **Track C — Your own data.** A public dataset you care about (for example from the National Institute of Statistics of Rwanda, Kaggle or your workplace — anonymised). Agree the target with a classmate or mentor first.

Whatever the track, follow the same seven parts and write one report.

Track A starter

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 600
district = rng.choice(["Gasabo", "Kicukiro", "Nyarugenge"], n, p=[0.45, 0.35, 0.2])
bedrooms = rng.integers(1, 5, n)
size_m2 = (bedrooms * 28 + rng.normal(20, 12, n)).clip(25).round()
km_to_centre = rng.uniform(0.5, 15, n).round(1)
furnished = rng.random(n) < 0.3
base = {"Gasabo": 120_000, "Kicukiro": 90_000, "Nyarugenge": 100_000}
rent = (np.array([base[d] for d in district]) + 2_600 * size_m2 - 6_000 * km_to_centre
        + np.where(furnished, 60_000, 0) + rng.normal(0, 25_000, n)).round(-3)
homes = pd.DataFrame({"district": district, "bedrooms": bedrooms, "size_m2": size_m2,
                      "km_to_centre": km_to_centre, "furnished": furnished, "rent_rwf": rent})
print(homes.shape)
homes.head()
```

Part 1 — Make it realistic and explore

```
rng = np.random.default_rng(5)
data = homes.copy()
data.loc[rng.choice(len(data), 30, replace=False), "size_m2"] = np.nan
data.loc[rng.choice(len(data), 15, replace=False), "district"] = np.nan
print(data.isna().sum())
print(data.groupby("district")["rent_rwf"].median())
```

Add at least two charts and three sentences of findings.

Part 2 — Lock away the test set and build the pipeline

```
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

X = data.drop(columns="rent_rwf")
y = data["rent_rwf"]
X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=11)

prepare = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()),
     ["bedrooms", "size_m2", "km_to_centre"]),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), ["district"]),
    ("flag", "passthrough", ["furnished"]),
])
```

Part 3 — Compare models with cross-validation

```
candidates = {
    "Baseline": DummyRegressor(),
    "Linear regression": LinearRegression(),
    "Ridge": Ridge(alpha=1.0),
    "Random forest": RandomForestRegressor(300, min_samples_leaf=3, random_state=11,
n_jobs=-1),
    "Gradient boosting": GradientBoostingRegressor(random_state=11),
}
for name, model in candidates.items():
    pipe = Pipeline([("prepare", prepare), ("model", model)])
    mae = -cross_val_score(pipe, X_dev, y_dev, cv=5, scoring="neg_mean_absolute_error")
    print(f"{name:18s} MAE {mae.mean():8.0f} +/- {mae.std():6.0f}")
```

The linear models should lead (MAE around 22,000–23,000 RWF), the tree ensembles follow, and the baseline is far behind (about 85,000). Explain **why** linear models suit this data.

Part 4 — Tune and test once

```
pipe = Pipeline([("prepare", prepare), ("model", Ridge())])
search = GridSearchCV(pipe, {"model__alpha": [0.01, 0.1, 1, 10, 100],
                             "prepare__num__simpleimputer__strategy": ["median", "mean"]},
                      cv=5, scoring="neg_mean_absolute_error")
search.fit(X_dev, y_dev)
print(search.best_params_, "CV MAE:", round(-search.best_score_))

from sklearn.metrics import mean_absolute_error, r2_score
final = search.best_estimator_
pred = final.predict(X_test)
print("Test MAE:", round(mean_absolute_error(y_test, pred)), " Test R²:",
round(r2_score(y_test, pred), 3))
```

Part 5 — Check errors by group

```
errors = pd.DataFrame({"district": X_test["district"].fillna("missing"),
                       "abs_error": (y_test - pred).abs()})
print(errors.groupby("district")["abs_error"].agg(["count", "mean"]).round(0))
```

Is the model much worse for any district? For homes with missing size? Say what you would do about it.

Part 6 — Save and document

```
import joblib, json, sklearn
joblib.dump(final, "rent_model_v1.joblib")
card = {"name": "Kigali rent estimate", "version": "v1",
        "sklearn_version": sklearn.__version__,
        "test_mae_rwf": round(mean_absolute_error(y_test, pred)),
        "intended_use": "Guide price shown to visitors of a property website",
        "limits": "Simulated practice data; no information on building quality, water or
security"}
json.dump(card, open("rent_model_v1_card.json", "w"), indent=2)
print(json.load(open("rent_model_v1_card.json")))
```

Part 7 — Report (two pages)

1. **Problem and data** — target, features, data problems found.
2. **Method** — pipeline steps, models compared, cross-validation, tuning.
3. **Results** — CV table, final test MAE and R², a predicted-vs-real chart.
4. **Errors and fairness** — results by group, and what you would change.
5. **Deployment plan** — saved file, model card, what you would monitor and when you would retrain.
6. **Limits and next steps.**

Marking guide (self-check)

Area	Excellent work
Data	Problems found and handled inside a pipeline
Method	Several models compared with cross-validation, one tuned
Honesty	Test set used once; baseline reported
Responsibility	Errors checked by group; limits stated
Engineering	Pipeline saved, model card written, monitoring plan
Communication	Clear charts and plain-English conclusions

Getting your certificate

1. Complete **every lesson** in the course (the progress bar shows 100%).
2. The **final exam** then unlocks on the course page: 60 questions (single answer, multiple answers, true/false and fill in the blank), 90 minutes, 60% to pass, 3 attempts. The exam opens in full screen; leaving the exam screen is counted, and the third time ends the exam automatically.
3. Pass the exam and your **WAMUGII Certificate** appears on the course page, showing your exam score. Add it — and a link to your final project notebook — to your CV and LinkedIn.

What you can do now

You can take a real table of data and turn it into a trustworthy model: prepare it in a pipeline, choose among linear models, k-NN, trees, forests, boosting, SVMs and Naive Bayes, evaluate with cross-validation, precision, recall and AUC, tune, find groups with clustering and PCA, check fairness, and deploy with a model card. Next in the WAMUGII AI Program: **deep learning**, where neural networks learn from images, sound and text.

Remember this: Professional machine learning is a disciplined process — honest evaluation, clear communication and responsibility matter as much as the algorithm.

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–7 for your chosen track in one notebook and share the notebook link with your instructor in the lesson Q&A below. Challenge: add one engineered feature and show, with cross-validation, whether it helps.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is the correct order for an honest project?
   - A. Train on everything, then test on the training data
   - B. Tune on the test set, then split
   - C. Split off the test set, develop with cross-validation, test once
   - D. Test, tune, split
2. Why are preparation steps inside the Pipeline in the final project?
   - A. Because models need text
   - B. To remove the target
   - C. So they are learned from training folds only and saved with the model
   - D. To make the code longer
3. Why check the errors by district?
   - A. To find groups the model serves badly
   - B. It is not useful
   - C. To increase R^2
   - D. Because the baseline needs it
4. Why might linear models win on the rent data?
   - A. Rents were built from a mostly straight-line rule
   - B. The data has no numbers
   - C. Trees cannot do regression
   - D. Linear models always win
5. What does the final exam require to pass?
   - A. 60%
   - B. 100%
   - C. 50%
   - D. 80%
6. What should a model card include?
   - A. Only the file name
   - B. Passwords
   - C. The test set
   - D. Intended use, data, performance and limits

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.